

Aula 1 — O que é back-end, Ruby e o primeiro Rails

Duração: ~3h · **Você sai daqui com:** uma API respondendo `GET /api/v1/status`. **Gabarito:** `cd ~/capitacao-gabarito && git checkout aula-01`

Pré-requisitos instalados em `00-preparacao.md`.

1. Os dois lados

Na capacitação de front-end você aprendeu a fazer o que o usuário vê. Back-end é o outro lado.

Front-end	Back-end
A parte visível, que o usuário toca	A parte lógica, nos bastidores
HTML, CSS, JavaScript	Ruby, Python, Java, Go, Node
Roda no navegador ou no celular	Roda num servidor, longe do usuário
Se cair, a tela quebra	Se cair, nada funciona

O back-end guarda os dados, decide quem pode ver o quê, aplica as regras de negócio, autentica usuários e conversa com outros sistemas (e-mail, pagamento, mapas).

Por que isso não pode viver no front? Porque tudo que roda no navegador o usuário consegue ler e alterar. A validação de senha no front é conveniência; a que vale é a do back.

A analogia: o salão do restaurante é o front-end — mesa, cardápio, garçom. A cozinha é o back-end: ninguém entra, mas é lá que a comida sai. O pedido é a requisição, o prato é a resposta. Você não pede pra ver a receita; você pede o prato.

2. A rede, o mínimo necessário

O que é um servidor, afinal

Não é uma máquina especial. É um **programa esperando**: ele fica ligado, escutando numa porta, e responde ao que chegar ali. `bin/rails server` sobe esse programa na porta 3000 — na sua máquina ou numa VM na nuvem, é o mesmo programa.

IP e porta

IP é o endereço da máquina (`57.156.65.151`). **Porta** é a sala dentro dela: um número de 1 a 65535. Uma máquina tem milhares de portas, e um programa diferente pode escutar em cada uma.

Porta	Quem mora ali
22	SSH
80	HTTP
443	HTTPS
5432	Postgres
3000	Rails em desenvolvimento

`127.0.0.1` — o `localhost` — significa sempre "esta máquina aqui". Na Aula 4 vamos abrir e fechar portas na mão, no firewall da Azure.

DNS

Ninguém decora `57.156.65.151`. O **DNS** é a agenda telefônica da internet: você digita `api.seemxxiii.tech` e alguém pergunta ao DNS qual é o IP.

A resposta fica em cache por um tempo — o **TTL**. É por isso que apontar um domínio para outro servidor não vale na hora. Na Aula 4 você vai criar um desses registros.

Anatomia de uma URL

```
https :// api.seemxxiii.tech : 443 /api/v1/lectures ?page=2 #topo
|         |           |         |         |         |
esquema  host        porta   caminho  query  fragmento
```

- **Esquema** — o protocolo: `http`, `https`, `ssh`, `postgres`.
- **Porta** — opcional. `http` assume 80, `https` assume 443.
- **Fragmento** — nunca vai para o servidor; é só para o navegador.

É por isso que `localhost:3000` tem os dois pontos: a porta não é a padrão.

3. HTTP

Cliente é quem pede (navegador, app, outro servidor). Servidor é quem responde. **O cliente sempre começa a conversa** — o servidor nunca liga primeiro.

Uma **requisição** tem método, caminho, cabeçalhos e corpo. Uma **resposta** tem status, cabeçalhos e corpo.

Como ela é, por dentro

```
POST /api/v1/sessions HTTP/1.1      ← método, caminho, versão
Host: api.seemxxiii.tech             ]
Content-Type: application/json        | cabeçalhos
Accept: application/json              ]
                                     ← linha em branco separa
{"email": "ana@ufop.br", "password": "..."} ← corpo
```

É **texto puro**. HTTP é literalmente isso trafegando num cano.

Cabeçalhos

São **metadados**: informação *sobre* a mensagem, não a mensagem. Um par `chave: valor` por linha.

Cabeçalho	Para quê
<code>Content-Type</code>	Em que formato vai o corpo
<code>Accept</code>	Em que formato eu quero a resposta
<code>Authorization</code>	Quem sou eu — vai ser o nosso token, na Aula 3
<code>Set-Cookie</code> / <code>Cookie</code>	Como o navegador guarda estado
<code>User-Agent</code>	Que programa está chamando

`Content-Type` importa

O mesmo dado, dois formatos:

```
application/json      → {"email": "ana@ufop.br"}
application/x-www-form-urlencoded → email=ana%40ufop.br
```

Sem o cabeçalho, o servidor não sabe como ler o corpo. Esquecer isso no `curl` é o erro nº 1 de quem começa: vem `400` e a mensagem não ajuda em nada. Por isso todo `curl` deste material tem `-H 'Content-Type: application/json'`.

Os métodos

Método	Significa
<code>GET</code>	Me dá isso. Não muda nada.
<code>POST</code>	Cria isso.
<code>PUT</code> / <code>PATCH</code>	Altera isso.
<code>DELETE</code>	Apaga isso.

O método é uma promessa. Um `GET` que apaga registro é bug, não criatividade.

Os status

Faixa	Significa	Exemplos
2xx	Deu certo	200 ok, 201 criado
3xx	Foi pra outro lugar	301, 302
4xx	Você errou	400 pedido mal feito, 401 não autenticado, 403 sem permissão, 404 não existe, 422 dado inválido
5xx	Nós erramos	500 bug nosso

401 é "quem é você?". 403 é "sei quem você é, e você não pode". Vamos usar os dois na Aula 3.

HTTP não tem memória

Cada requisição começa do zero. O servidor esquece tudo entre uma e outra.

Guarde essa frase. Ela é o problema inteiro da Aula 3: se o servidor esquece tudo, como ele sabe que você já fez login?

Veja funcionando

```
curl -i https://api.seemxxiii.tech/api/v1/status
```

```
HTTP/2 200
content-type: application/json; charset=utf-8

{"status":"ok","service":"seem-backend"}
```

Isso é uma API real, no ar. É exatamente o que você vai construir.

4. API REST

API é a porta de entrada do sistema para outros programas. **REST** é um estilo de organizar essa porta: cada coisa do sistema é um **recurso**, com um endereço.

```
/api/v1/lectures    → as palestras
/api/v1/lectures/7  → a palestra 7
```

O que você faz com o recurso é o **verbo**, não o endereço:

GET	/api/v1/lectures	lista
POST	/api/v1/lectures	cria
GET	/api/v1/lectures/7	mostra uma
DELETE	/api/v1/lectures/7	apaga

`/criarPalestra` · `POST /lectures`

O `v1` no caminho é a versão. Quando a API mudar de forma incompatível, nasce uma `/v2` e a `/v1` continua funcionando — porque o app na loja do celular demora dias para atualizar.

5. Ruby, partindo do Python

Ruby foi criado por **Yukihiro Matsumoto (Matz)** em 1995, no Japão, com um objetivo declarado: *fazer o programador feliz*. É interpretada, dinâmica e orientada a objetos — como Python.

A tabela completa está em `ruby-para-pythonistas.md`. O essencial:

```
# Python                                # Ruby
def saudacao(nome, formal=False):      def saudacao(nome, formal: false)
  if formal:                            return "Prezado #{nome}" if formal
    return f"Prezado {nome}"           "Oi, #{nome}"
  return f"Oi, {nome}"                 end
```

Três coisas para reparar:

1. `end` fecha o bloco, em vez da indentação.
2. `if` **no fim da linha** — é o modificador, muito usado em Ruby.
3. **A última linha é o retorno**. Não precisa escrever `return`.

Tudo é objeto

Em Python você chama `len(x)`. Em Ruby, `x.length`. Não existe função solta: é sempre método de alguém. Até número é objeto — `3.times { puts "oi" }`.

Símbolos

```
:aluno  :email  :admin
```

Um símbolo é um texto que serve de **etiqueta**, não de conteúdo. O mesmo símbolo é sempre o mesmo objeto na memória. Não existe em Python — lá você usaria uma string.

Regra prática: **conteúdo que o usuário lê** → **string. Nome de coisa no código** → **símbolo**.

Blocos

Um bloco é um pedaço de código que você entrega para um método executar. É o `lambda` do Python, mas usado o tempo todo.

```
usuarios.each do |u|           # for u in usuarios
  puts u.nome
end

usuarios.map { |u| u.nome }     # [u.nome for u in usuarios]
usuarios.select { |u| u.ativo? } # [u for u in usuarios if u.ativo]
```

Chaves quando cabe numa linha, `do ... end` quando não cabe.

? e !

Sufixo	Significa	Exemplo
<code>?</code>	Devolve verdadeiro ou falso	<code>user.admin?</code>
<code>!</code>	A versão perigosa: estoura erro em vez de devolver <code>false</code>	<code>user.save!</code>

É convenção, não regra da linguagem. Mas todo mundo segue.

Argumentos nomeados, e o atalho do Ruby 3.1

```
def login(email:, password:, client: "mobile")
  end

login(email: "a@b.c", password: "x")
```

Os dois-pontos **depois** do nome tornam obrigatório nomear na chamada. E quando a variável tem o mesmo nome da chave, você omite o valor:

```
Result.new(success?: true, user:) # user: user
```

Isso aparece em todo service do projeto. **Não é erro de digitação.**

Struct

Quando você só quer agrupar valores, sem comportamento:

```
Result = Struct.new(:success?, :user, :errors, keyword_init: true)

r = Result.new(success?: true, user: ana)
r.success? # true
```

É o `namedtuple` / `dataclass` do Python. Todo service deste projeto devolve um desses.

Exceções

Python	Ruby
<code>try</code>	<code>begin</code>
<code>except</code>	<code>rescue</code>
<code>finally</code>	<code>ensure</code>
base para capturar: <code>Exception</code>	<code>StandardError</code>

```
def call
  arriscado
rescue StandardError => e
  Rails.error.report(e)
  nil
end
```

Dentro de um método o `begin` é implícito — dá para escrever só o `rescue` no fim. Criando o seu tipo de erro:

```
class InvalidToken < StandardError; end

raise InvalidToken if token_ruim?
```

Herde sempre de `StandardError`, nunca de `Exception`: capturar `Exception` pega até `Ctrl+C`.

Módulos e mixins

Ruby não tem herança múltipla. Tem **módulo**: um saco de métodos que você inclui numa classe.

```
class User < ApplicationRecord
  include EmailVerifiable
  include PasswordResetable
end
```

Vamos escrever esses dois módulos na Aula 2. O nome deles no mundo Rails é **concern**.

Dependências

Python	Ruby
<code>pip install</code>	<code>gem install</code>
<code>requirements.txt</code>	<code>Gemfile</code>
<code>venv</code> por projeto	<code>bundler</code> resolve tudo
<code>pip freeze</code> para travar	<code>Gemfile.lock</code> , gerado sozinho

O `Gemfile.lock` é o `requirements.txt` travado — só que automático e **sempre versionado**. Ele é a garantia de que a sua máquina e o servidor rodam exatamente as mesmas versões.

Prática rápida

```
irb
```

```
3.class          # Integer
"oi".class       # String
:oi.class        # Symbol
nil.class        # NilClass

5.times { |i| puts i }

usuario = { nome: "Ana", role: :admin }
usuario[:nome]

[1, 2, 3].map { |n| n * n }
```

6. Rails

Framework web em Ruby, criado por **David Heinemeier Hansson** em 2004 — extraído do Basecamp, um produto real. Traz tudo junto: rotas, banco, e-mail, filas, testes, deploy. Estamos na versão 8.

As duas leis

Convenção sobre configuração. Você não configura o óbvio, você segue o combinado. Classe `User`? Então a tabela é `users`, o arquivo é `user.rb`, a chave primária é `id`. Ninguém precisa dizer. Seguindo a convenção você escreve quase nada; brigando com ela, o dobro.

DRY — *Don't Repeat Yourself*. Cada regra existe num lugar só.

E uma atitude: **omakase**, "deixa comigo, chef". O Rails já escolheu as ferramentas por você. Você pode trocar, mas só troque quando tiver um motivo real. Isso é liberdade: sobra tempo pro problema de verdade.

Rails × Django

Django	Rails
<code>models.py</code>	<code>app/models/</code> , um arquivo por classe
<code>makemigrations</code> / <code>migrate</code>	<code>rails g migration</code> / <code>db:migrate</code>
<code>urls.py</code>	<code>config/routes.rb</code>
<code>views.py</code>	<code>app/controllers/</code>
Django ORM	ActiveRecord
<code>manage.py</code>	<code>bin/rails</code>

Quem vem de Flask ou FastAPI vai sentir mais diferença: lá você monta cada peça, aqui elas já vêm encaixadas.

MVC

- **Model** — os dados e as regras. Conversa com o banco.
- **View** — a tela. Na nossa API, é o JSON.
- **Controller** — recebe a requisição e decide o que fazer.
- **Rota** — o mapa: este endereço vai para aquele controller.

O caminho é sempre: **rota** → **controller** → **model** → **resposta**.

Zeitwerk: o nome diz o caminho

Você nunca escreve `require` no Rails. Por quê?

O **Zeitwerk** carrega a classe no instante em que você a menciona, e descobre o arquivo pelo **nome da classe**:

```
Api::V1::StatusController → app/controllers/api/v1/status_controller.rb
```

Errou o caminho, a classe simplesmente não existe. Não é questão de estilo — é o mecanismo que faz "convenção sobre configuração" funcionar de verdade.

Os três ambientes

Ambiente	Onde	Como se comporta
<code>development</code>	sua máquina	recarrega o código a cada requisição, log verboso, erro na tela
<code>test</code>	os testes	banco separado, limpo a cada teste
<code>production</code>	o servidor	código congelado, log enxuto, erro genérico

Cada um tem um arquivo em `config/environments/`. `Rails.env` diz onde você está.

É assim que o e-mail abre no navegador em desenvolvimento e sai por SMTP em produção — mesmo código, ambientes diferentes.

7. Mão na massa

Criar o projeto

```
rails new automic_auth_api --api -d postgresql \
  --skip-action-mailbox --skip-action-text --skip-active-storage \
  --skip-jbuilder --skip-action-cable
cd automic_auth_api
```

- `--api` — sem HTML, sem CSS, sem sessão de navegador. Só JSON.
- `-d postgresql` — o mesmo banco que usamos em produção.
- Os `--skip` tiram peças que não vamos usar. Cada peça a menos é uma peça a menos para dar problema.

Subir o banco

Crie o `compose.yaml` (ou copie o deste repositório) e rode:

```
docker compose up -d
docker compose ps      # tem que aparecer "healthy"
```

Se der `address already in use`, você já tem um Postgres na máquina ocupando a 5432. Rode `DB_PORT=5433 docker compose up -d` e exporte `DB_PORT=5433` no shell.

Preparar e subir a aplicação

```
bin/rails db:prepare
bin/rails server
```

Abra `http://localhost:3000/up`. Verde é a aplicação de pé.

O tour das pastas

Pasta	O que tem
<code>app/</code>	O seu código. É onde você passa 90% do tempo.
<code>config/</code>	Rotas, banco, ambientes, credenciais
<code>db/</code>	Migrations e o retrato atual do banco (<code>schema.rb</code>)
<code>test/</code>	Os testes
<code>bin/</code>	Os comandos: <code>rails</code> , <code>setup</code> , <code>dev</code>
<code>Gemfile</code>	As dependências

Os comandos do dia a dia

```
bin/rails server      # sobe a aplicação
bin/rails console     # um irb com o seu projeto carregado dentro
bin/rails routes      # todas as rotas que existem
bin/rails generate     # cria arquivos seguindo a convenção
bin/rails db:migrate  # aplica as mudanças de banco
bin/rails test         # roda os testes
```

O **console** é a ferramenta mais subestimada do Rails: dá para criar usuário, rodar query e testar método sem escrever um arquivo.

8. A primeira rota

`config/routes.rb`:

```
Rails.application.routes.draw do
  get "up" => "rails/health#show", as: :rails_health_check

  namespace :api do
    namespace :v1 do
      get "status", to: "status#show"
    end
  end
end
```

`app/controllers/api/v1/status_controller.rb`:

```

module Api
  module V1
    class StatusController < ApplicationController
      def show
        render json: {
          status: "ok",
          service: "atomic-auth-api",
          environment: Rails.env
        }
      end
    end
  end
end

```

Repare na convenção: `namespace :api` + `namespace :v1` + `status#show` implica exatamente o arquivo `app/controllers/api/v1/status_controller.rb`, classe `Api::V1::StatusController`, método `show`. Ninguém configurou isso.

Teste

`test/controllers/api/v1/status_controller_test.rb`:

```

require "test_helper"

class Api::V1::StatusControllerTest < ActionDispatch::IntegrationTest
  test "responde ok" do
    get "/api/v1/status"

    assert_response :ok
    assert_equal "ok", response.parsed_body["status"]
  end
end

```

```
bin/rails test
```

Exercício da aula

Onde você trabalha: no **seu** projeto, que você cria no passo 1. O repositório da capacitação é o **gabarito** — abra para consultar, não para escrever.

1. Crie o seu projeto

```
cd ~
rails new automic_auth_api --api -d postgresql \
  --skip-action-mailbox --skip-action-text --skip-active-storage \
  --skip-jbuilder --skip-action-cable
cd automic_auth_api
```

2. Publique no seu GitHub

Não é opcional: na Aula 4, o deploy automático precisa de um repositório **seu**.

```
git add -A && git commit -m "Projeto inicial"
gh repo create automic_auth_api --private --source=. --push
```

Sem o `gh` instalado, crie o repositório pelo site e depois:

```
git remote add origin git@github.com:SEU-USUARIO/automic_auth_api.git
git push -u origin main
```

3. Suba o banco

Crie um arquivo `compose.yaml` na raiz do projeto:

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: automic
      POSTGRES_PASSWORD: automic
      POSTGRES_DB: automic_auth_api_development
    ports:
      - "${DB_PORT:-5432}:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U automic"]
      interval: 5s
      retries: 10

volumes:
  pgdata:
```

E em `config/database.yml`, dentro do bloco `default: &default`, acrescente:

```
host: <%= ENV.fetch("DB_HOST", "localhost") %>
port: <%= ENV.fetch("DB_PORT", 5432) %>
username: <%= ENV.fetch("DB_USER", "automic") %>
password: <%= ENV.fetch("DB_PASSWORD", "automic") %>
```

```
docker compose up -d
docker compose ps      # tem que aparecer "healthy"
```

address already in use? Você já tem um Postgres na 5432. Rode `DB_PORT=5433 docker compose up -d` e `export DB_PORT=5433` no shell.

4. Suba a aplicação

```
bin/rails db:prepare
bin/rails server
```

Abra `http://localhost:3000/up`. Verde é a aplicação de pé.

5. Escreva a rota

Em `config/routes.rb`, dentro do `Rails.application.routes.draw do`:

```
namespace :api do
  namespace :v1 do
    get "status", to: "status#show"
  end
end
```

Crie o arquivo `app/controllers/api/v1/status_controller.rb` — **o caminho tem que ser exatamente esse**, é o Zeitwerk:

```
module Api
  module V1
    class StatusController < ApplicationController
      def show
        render json: {
          status: "ok",
          service: "automic-auth-api",
          environment: Rails.env
        }
      end
    end
  end
end
```

6. Confira

```
bin/rails routes -g api      # a sua rota tem que aparecer
curl localhost:3000/api/v1/status
```

Depois monte a mesma requisição no Insomnia e confira o `200`.

7. Escreva o teste

Crie `test/controllers/api/v1/status_controller_test.rb`:

```
require "test_helper"

class Api::V1::StatusControllerTest < ActionDispatch::IntegrationTest
  test "responde ok" do
    get "/api/v1/status"

    assert_response :ok
    assert_equal "ok", response.parsed_body["status"]
  end
end
```

```
bin/rails test
```

8. Commit

```
git add -A && git commit -m "Rota de status" && git push
```

Bônus: faça a rota devolver também `RUBY_VERSION` e `Rails.version`.

Travou? Abra o mesmo arquivo no gabarito, entenda o que está diferente, e conserte o seu:

```
cd ~/capacitacao-gabarito && git checkout aula-01
cat app/controllers/api/v1/status_controller.rb
```

Recapitulando

- Back-end é a cozinha: regra, dado e permissão.
- HTTP é o idioma; verbo, status e JSON são o vocabulário.
- HTTP não tem memória — segure essa ponta até a Aula 3.
- Ruby é Python com outra sintaxe e mais açúcar.
- Rails decide o óbvio por você. Siga a convenção.

Na próxima: o banco de dados entra em cena, e a API ganha um `User` com senha de verdade.